

Instead, you can put the multiple values into a tuple and check for whether a variable's value exists in that tuple using the `in` operator, as in the following example:

```
>>> # Pythonic Example
>>> spam = 'cat'
>>> spam in ('cat', 'dog', 'moose')
True
```

Not only is this idiom easier to understand, it's also slightly faster, according to `timeit`.

Summary

All programming languages have their own idioms and best practices. This chapter focuses on the particular ways that Python programmers have come to write “pythonic” code to make the best use of Python's syntax.

At the core of pythonic code are the 20 aphorisms from the Zen of Python, which are rough guidelines for writing Python. These aphorisms are opinions and not strictly necessary for writing Python code, but they are good to keep in mind.

Python's significant indentation (not to be confused with significant whitespace) provokes the most protest from new Python programmers. Although almost all programming languages commonly use indentation to make code readable, Python requires it in place of the more typical braces that other languages use.

Although many Python programmers use the `range(len())` convention for `for` loops, the `enumerate()` function offers a cleaner approach to getting the index and value while iterating over a sequence. Similarly, the `with` statement is a cleaner and less bug-prone way to handle files compared to calling `open()` and `close()` manually. The `with` statement ensures that `close()` gets called whenever the execution moves outside the `with` statement's block.

Python has had several ways to interpolate strings. The original way was to use the `%s` conversion specifier to mark where strings should be included in the original string. The modern way as of Python 3.6 is to use f-strings.